# ReleaseGate

Evidence-Driven AI Data Release Governance Agent

Final Technical Architecture & Engineering Blueprint

**VERSION 1.0**

## 1. Project definition

We treat ReleaseGate as an independent software product, not just a course assignment. The objective is to build a working AI agent system that demonstrates secure tool execution, independent AI reasoning, human authorization, reliable background processing and production-grade engineering. These decisions should be frozen before development.

### ReleaseGate - Evidence-Driven AI Data Release Governance Agent

A hybrid AI system that inspects data locally, evaluates risks, proposes remediation, obtains human approval and safely executes authorized publication.

### Core operating principles

- AI can investigate, analyze and recommend.
- AI cannot independently authorize a release.
- Every publication must have human authorization.
- Authorization applies to an exact dataset version, not merely a dataset name.
- An independent reviewer must challenge the primary agent.
- Every tool execution must be recorded.
- Repeated requests must not create duplicate operations.
- Administrators must be able to stop all execution immediately.
- Original datasets must remain unchanged.
- Sensitive source data must remain inside the local processing environment.

### Selected operating model

| Decision | Final selection |
|---|---|
| Autonomy | Hybrid |
| Deployment | Local processing + cloud dashboard |
| Frontend | React + Vite + TypeScript |
| Backend | Python + FastAPI |
| Agent orchestration | Custom bounded agent loop |
| Tool communication | MCP |
| Database | PostgreSQL |
| Local file processing | Python |

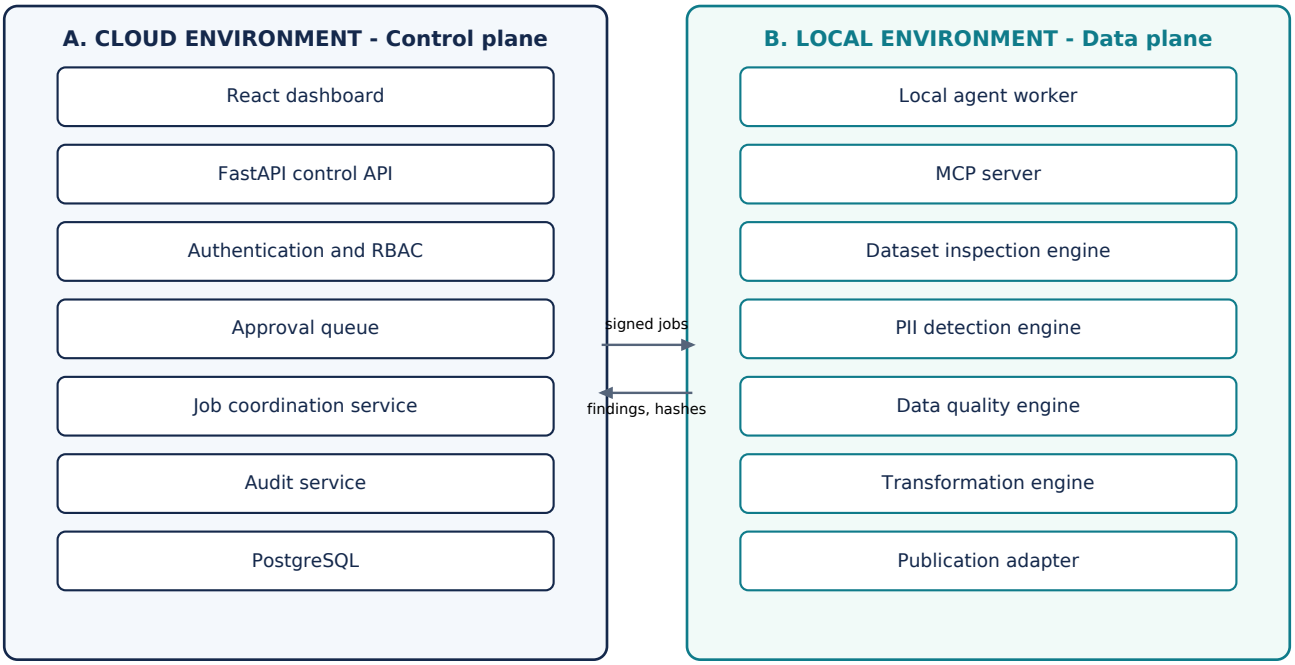
| Decision | Final selection |
|---|---|
| AI reviewer | Independent model client |
| Approval | Human-in-the-loop |
| Job scheduling | APScheduler initially |
| Deployment packaging | Docker |

# 2. System architecture

The most important architectural decision is to separate the cloud control plane from the local data processing plane. This is what makes ReleaseGate genuinely hybrid.

## 2.1 High-level architecture

The architecture has two environments.




**A. Cloud environment (control plane).** Manages users, approvals, policies, agent job coordination and audit records. It hosts the React dashboard, FastAPI control API, PostgreSQL database, authentication and RBAC, approval queue, audit service and job coordination service.

**B. Local environment (data plane).** Processes the actual files. It hosts the local agent worker, MCP server, dataset inspection engine, PII detection engine, data quality engine, local encrypted file storage, transformation engine and publication adapter.

**Data flow rule:** The cloud environment receives dataset metadata, findings, hashes, approval information and sanitized evidence. Raw files and sensitive row-level content remain local.

The local worker initiates outbound authenticated communication with the cloud. Do not expose the local MCP server directly to the public internet.

## 2.2 Detailed component architecture

Request flow, from the human user to publication:

| Step | Flow |
|---|---|
| 1 | Human user to React dashboard to FastAPI control API. |

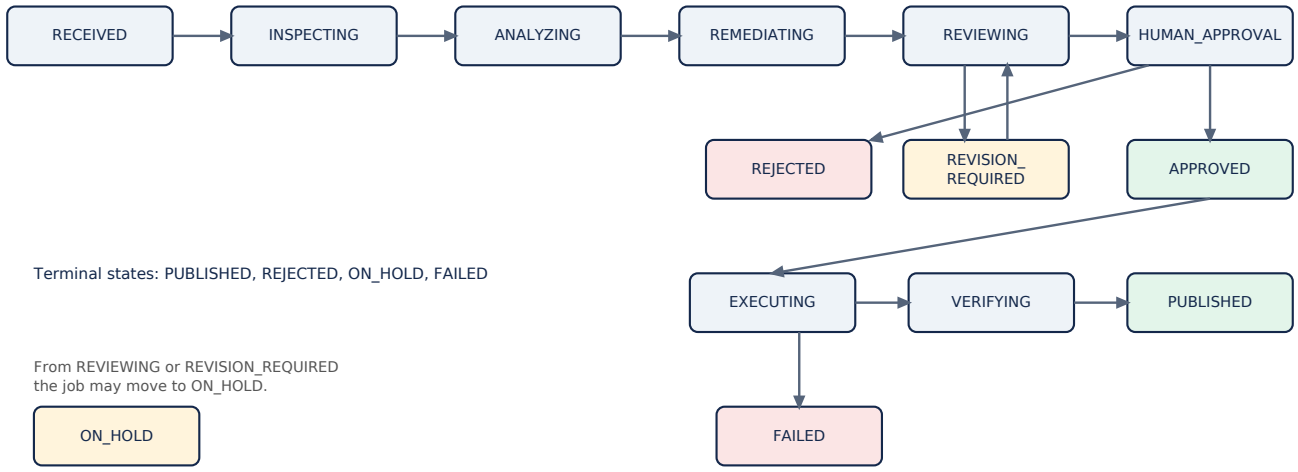
| Step | Flow |
|---|---|
| 2 | API uses authentication and RBAC, PostgreSQL, and the job coordinator. |
| 3 | Job coordinator sends a **signed job request** to the local worker. |
| 4 | Local worker runs the bounded agent orchestrator, which calls the primary model client and the local MCP server. |
| 5 | MCP server exposes dataset inspection, PII detection, data quality, policy engine, transformation engine and publication adapter. |
| 6 | Independent reviewer challenges the report and sends the result to the API and the human approval queue (stored in PostgreSQL). |
| 7 | API sends a **signed approved command** to the local worker, which calls the MCP publication tool. |
| 8 | Publication verification writes the result to the database. |
| 9 | The agent, API and MCP server all emit events to the audit event service. |

One important implementation detail: the model never directly connects to PostgreSQL or accesses the filesystem. It requests operations through validated tools. The cloud API coordinates the process, while the local execution service enforces the authorization before publishing.

# 3. Agent architecture

This is the heart of the capstone. A custom bounded agent loop is recommended instead of allowing an LLM to execute an unlimited sequence of actions.

## 3.1 Agent lifecycle




### Agent responsibilities

| Component | Responsibility |
|---|---|
| Planner | Determines the next required inspection step |
| Primary agent | Interprets evidence and recommends action |
| Tool validator | Validates tool names and arguments |
| Tool executor | Invokes authorized MCP tools |
| Reviewer | Independently challenges findings and recommendations |
| Approval manager | Collects human authorization |
| Execution manager | Executes approved operations |

| Component | Responsibility |
|---|---|
| Audit manager | Records all important events |

## 3.2 Bounded agent loop

The agent must have explicit limits. Recommended:

- Maximum 12 tool calls per investigation.
- Maximum 8 reasoning iterations.
- One controlled rewrite after reviewer rejection.
- Maximum execution retry count of 3, with backoff.
- Fixed model timeout.
- Token and monetary budget per job.
- Explicit stop reasons.

Possible stop reasons:

```
COMPLETED
HUMAN_APPROVAL_REQUIRED
REVIEW_REJECTED
MAX_ITERATIONS_REACHED
TOOL_VALIDATION_FAILED
MODEL_TIMEOUT
POLICY_BLOCKED
EXECUTION_FAILED
SYSTEM_PAUSED
```

The system must never treat a model timeout or missing response as permission to continue.

## 3.3 Two-model review architecture

| Role | Name | Function |
|---|---|---|
| Primary agent | Investigator | Receives dataset metadata and inspection results. Produces findings, risk levels and proposed transformations. |
| Independent reviewer | Challenger | Receives the original evidence, applicable policies and the primary agent's report. Independently evaluates the conclusions. |
| Human reviewer | Final authority | Examines the findings, proposed output and review decision. Explicitly approves or rejects the release. |

The reviewer should use a different model or at least an independently configured model session. It must not simply repeat the primary agent's reasoning. Reviewer output should be structured:

```
{
  "decision": "REJECT",
  "risk_score": 86,
  "issues": [
    {
      "finding_id": "F-102",
      "reason": "Sensitive identifier remains in output",
      "severity": "HIGH"
    }
  ],
  "required_changes": [
    "Remove or mask the identifier"
  ],
  "confidence": 0.94
}
```

The example is illustrative. In production, every issue must reference actual inspection evidence.

**Important:** AI review cannot certify that a dataset is legally safe. Deterministic checks and human judgment remain essential.

# 4. Module breakdown

ReleaseGate should be developed as a modular monorepo with the following structure:

```
releasegate/
|
|-- apps/
|   |-- web/
|   |   |-- src/
|   |   |   |-- components/  pages/  layouts/
|   |   |   |-- services/    hooks/  types/
|   |   `-- package.json
|   `-- api/
|       |-- app/
|       |   |-- auth/  users/  datasets/  releases/
|       |   |-- approvals/  audit/  jobs/
|       |   `-- main.py
|       `-- requirements.txt
|
|-- agent/
|   |-- orchestrator/  planner/  primary_agent/
|   |-- reviewer/  tool_validator/  execution/
|   `-- model_client/
|
|-- mcp-server/
|   |-- tools/
|   |   |-- inspection.py     privacy.py
|   |   |-- quality.py        policy.py
|   |   |-- transformation.py approval.py
|   |   `-- publication.py
|   |-- schemas/
|   |-- server.py
|   `-- permissions.py
|
|-- engines/
|   |-- dataset_profiler/  pii_detector/
|   |-- quality_validator/  policy_engine/
|   `-- transformation_engine/  publication_verifier/
|
|-- local-worker/
|   |-- worker.py  cloud_client.py  secure_storage.py
|   `-- job_runner.py  health.py
|
|-- shared/       (schemas/  enums/  contracts/)
|-- tests/        (unit/  integration/  security/  end_to_end/)
|-- infrastructure/ (docker/  migrations/  deployment/)
|-- docs/         (architecture.md  security-model.md
|                  mcp-contracts.md  threat-model.md)
|
|-- docker-compose.yml
|-- .env.example
|-- .gitignore
`-- README.md
```

### Module responsibilities

| Module | What it does |
|---|---|
| web | Dashboard and user interaction |
| api | Authentication, APIs, approvals and job coordination |
| agent | AI planning and reasoning |
| mcp-server | Controlled tool interface |
| engines | Deterministic data analysis |
| local-worker | Local execution and communication |
| shared | Common schemas and contracts |
| tests | Automated verification |

Keep the engines independent of the LLM. For example, PII detection must work even if the model provider is unavailable.

# 5. Database architecture

Use PostgreSQL for cloud-side application state. Local files live in encrypted local storage. The database stores their identifiers, hashes and metadata, not their sensitive contents.

## 5.1 Database entities

Thirteen primary tables are recommended.

| Table | Purpose |
|---|---|
| users | User accounts |
| organizations | Organization or workspace |
| memberships | User roles and permissions |
| datasets | Dataset metadata |
| dataset_versions | Immutable version records |
| inspection_runs | Inspection execution records |
| findings | Detected problems and evidence references |
| release_requests | Proposed releases |
| review_results | Independent reviewer decisions |
| approval_requests | Human authorization requests |
| execution_records | Publication attempts and results |
| audit_events | Append-only activity history |
| scheduled_jobs | Periodic monitoring jobs |

Supporting tables can be introduced later for policy versions, transformation plans, idempotency keys, worker registrations and tool execution records.

## 5.2 Entity relationships

| Parent | Relationship | Child |
|---|---|---|
| organizations | contains (1 to many) | memberships |
| users | belongs (1 to many) | memberships |
| organizations | owns | datasets |
| datasets | has | dataset_versions |
| dataset_versions | inspected by | inspection_runs |
| inspection_runs | produces | findings |
| datasets / dataset_versions | requested / referenced by | release_requests |
| release_requests | reviewed by | review_results |
| release_requests | requires | approval_requests |
| release_requests | executes | execution_records |
| users | decides | approval_requests |
| users | performs | audit_events |
| organizations | schedules | scheduled_jobs |

## 5.3 Important database fields

These are the core structures to implement.

### Users

| Field | Type |
|---|---|
| id | UUID, PK |
| email | VARCHAR, unique |
| password_hash | TEXT |
| full_name | VARCHAR |
| status | VARCHAR |
| created_at | TIMESTAMPTZ |

### Datasets

| Field | Type |
|---|---|
| id | UUID, PK |
| organization_id | UUID, FK |
| name | VARCHAR |
| description | TEXT |
| current_version_id | UUID, nullable |
| created_by | UUID, FK |
| created_at | TIMESTAMPTZ |

### Dataset versions

| Field | Type |
|---|---|
| id | UUID, PK |
| dataset_id | UUID, FK |
| version_number | INTEGER |
| content_hash | CHAR(64) |
| file_format | VARCHAR |
| file_size | BIGINT |
| local_artifact_ref | TEXT |
| metadata | JSONB |
| created_at | TIMESTAMPTZ |

### Inspection runs

| Field | Type |
|---|---|
| id | UUID, PK |
| dataset_version_id | UUID, FK |
| status | VARCHAR |
| risk_score | INTEGER |
| started_at | TIMESTAMPTZ |
| completed_at | TIMESTAMPTZ |
| agent_version | VARCHAR |
| policy_version | VARCHAR |

### Findings

| Field | Type |
|---|---|
| id | UUID, PK |
| inspection_run_id | UUID, FK |
| category | VARCHAR |
| severity | VARCHAR |
| description | TEXT |
| evidence_ref | JSONB |
| remediation | TEXT |
| status | VARCHAR |

### Release requests

| Field | Type |
|---|---|
| id | UUID, PK |
| dataset_version_id | UUID, FK |
| inspection_run_id | UUID, FK |
| proposed_output_hash | CHAR(64), nullable |
| status | VARCHAR |
| destination_id | UUID |
| reviewer_status | VARCHAR |
| created_by | UUID |
| created_at | TIMESTAMPTZ |

### Approval requests

| Field | Type |
|---|---|
| id | UUID, PK |
| release_request_id | UUID, FK |
| artifact_hash | CHAR(64) |
| policy_version | VARCHAR |
| status | VARCHAR |
| requested_at | TIMESTAMPTZ |
| decided_by | UUID, nullable |
| decided_at | TIMESTAMPTZ, nullable |
| rejection_reason | TEXT, nullable |

### Execution records

| Field | Type |
|---|---|
| id | UUID, PK |
| release_request_id | UUID, FK |
| idempotency_key | VARCHAR, unique |
| artifact_hash | CHAR(64) |
| status | VARCHAR |
| destination_receipt | JSONB |
| attempt_count | INTEGER |
| started_at | TIMESTAMPTZ |
| completed_at | TIMESTAMPTZ |

### Audit events

| Field | Type |
|---|---|
| id | UUID, PK |
| organization_id | UUID, FK |
| actor_type | VARCHAR |
| actor_id | UUID, nullable |
| action | VARCHAR |
| resource_type | VARCHAR |
| resource_id | UUID |
| event_payload | JSONB |
| previous_event_hash | CHAR(64) |
| event_hash | CHAR(64) |
| created_at | TIMESTAMPTZ |

### Database rules that must be enforced

- A dataset version cannot be modified after registration.
- A release request must reference an existing dataset version.

- An approval must reference the exact proposed output hash.
- Any transformation after approval invalidates the approval.
- A rejected release cannot be executed.
- An execution key cannot be reused for a different operation.
- Audit events cannot be updated or deleted by application users.
- Only authorized roles can approve releases.
- An approver cannot approve a release they submitted, by default.

For actual enforcement, use database constraints, restricted database roles, transactions and application-level authorization. Merely having these rules in documentation is insufficient.

### 5.4 Risk assessment model

Do not let an LLM arbitrarily invent the final risk score. Use a deterministic scoring system, with AI helping classify and explain findings. Initial model:

| Category | Weight |
|---|---|
| Sensitive information | 35 |
| Data quality | 20 |
| Licensing and provenance | 20 |
| Metadata completeness | 10 |
| Policy violations | 15 |
| **Total** | **100** |

Suggested risk bands:

| Score | Band |
|---|---|
| 0-19 | Low |
| 20-39 | Moderate |
| 40-69 | High |
| 70-100 | Critical |

These are initial engineering thresholds, not universal compliance standards. Any critical policy violation should independently block publication regardless of the aggregate score.

## 6. MCP server and tool contracts

This is one of the most important parts of the course. We should build a genuine MCP server, independently runnable and usable by an MCP-compatible client. The server exposes tools through typed input and output schemas. The LLM never receives unrestricted filesystem or database access.

### 6.1 MCP tool registry

Twelve tools are recommended. The first four are defined below.

| Tool | Input | Output |
|---|---|---|
| inspect_dataset | Dataset version ID | Dataset profile |
| detect_sensitive_data | Dataset version ID | Detection findings |
| validate_data_quality | Dataset version ID | Quality report |
| check_release_policy | Release ID | Policy evaluation |

**Note on completeness:** the source text supplied for this document ends partway through the tool registry (at *generate_risk_report*). The remaining tool definitions, and any later sections, are not included in this v1.0 file and should be added when the full text is available.